

BoolFellas: Implementation of an Information Retrieval System for Medical Scientific Documents

Progress Report

Furkan Safa Altunyuva
e275037@metu.edu.tr

Waleed Shanaa
e279012@metu.edu.tr

Abstract

This project aims to design and implement an Information Retrieval (IR) system for biomedical scientific literature using the Highwire Press collection from the TREC Genomics Tracks [1]. The system will support efficient indexing, searching, and ranking of medical articles by combining classical and neural IR techniques. Following initial feedback, **the implementation has been revised to use PyTerrier [4] as the core IR framework instead of a from scratch Java implementation, and the project has been extended with a neural re-ranking stage using a biomedical language model.**

1 Introduction

The primary problem addressed in this project is the difficulty of locating specific, relevant information within the medical scientific literature. In the modern research process, the high volume of published articles makes manual discovery inefficient. Researchers often find themselves overwhelmed by information overload, where generic search tools return thousands of results, many of which lack the required depth or clinical relevance.

The motivation for this project comes from the need for precision in academic and medical research. Unlike general web searches, missing a single relevant document in a medical context can lead to incomplete literature reviews. It is particularly interesting to explore how different ranking algorithms, such as the Vector Space Model and BM25, handle the highly specific and technical vocabulary of the biomedical field.

Based on the feedback, the project scope has been revised in several ways. First, **the system is now built on PyTerrier rather than a custom Java implementation.** Second, **a neural re-ranking stage based on biomedical language models, such as BioBERT [3] and PubMedBERT[2] has been added.** Third, **the indexing layer has been extended with field indexing alongside the existing positional index.** The system additionally supports wildcard queries, proximity operators, MeSH-based query expansion, **relevance feedback, and snippet generation.** Implementation details for each component are discussed in Section 2.

2 Methodology/System Architecture

Our system is built as a multi-layer IR pipeline using PyTerrier. Starting from an inverted index over the Highwire corpus, queries pass through preprocessing and expansion, a classical retrieval stage (BM25 or TF-IDF), and finally a neural re-ranking stage. Rather than evaluating a single model in isolation, we implement multiple ranking techniques and compare them against the same TREC qrels to identify the most effective approach.

Figure 1 shows the layered system architecture and Figure 2 illustrates the data flow for a single query. The UML class diagram in Figure 3 details the component interfaces and their relationships.

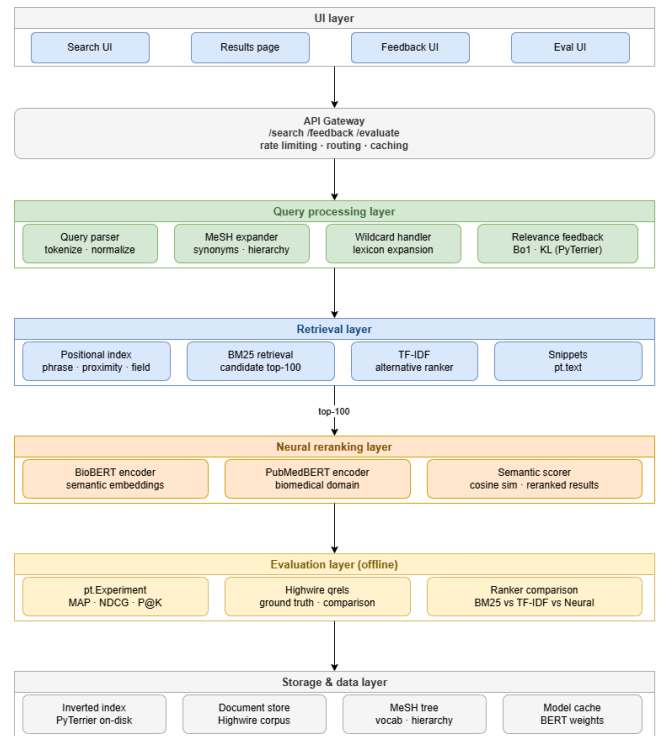


Figure 1. Layered system architecture: UI, query processing, retrieval, neural re-ranking, evaluation, and storage layers.

Data. The corpus is the Highwire Press medical document collection, a benchmark dataset from the TREC Genomics Tracks 2006–2007 consisting of 162,259 full-text scientific articles from 49 biomedical journals [1]. Each document

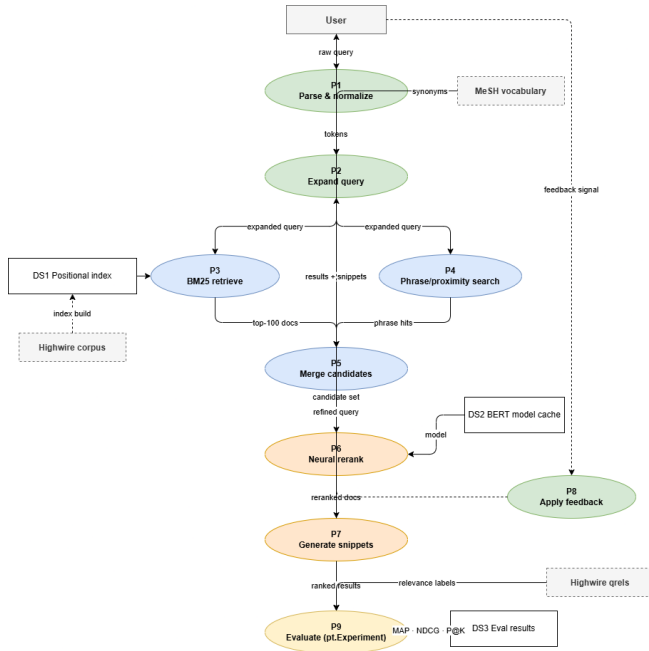


Figure 2. Data-flow diagram for query processing. Green nodes are query-side processes, blue nodes are retrieval processes, orange nodes are re-ranking and output processes.

contains structured fields including *doc_id*, *journal*, *title*, and *text spans*. Relevance judgments from 64 TREC queries (28 from 2006, 36 from 2007) serve as ground truth.

Index. Preprocessing includes tokenization, lowercase normalization, stop-word removal, and Porter stemming, applied via Terrier’s default indexing pipeline. Two indices are maintained, a *positional index* (built with *blocks=True*) that stores token positions to support phrase and proximity queries, and a *field index* covering *title*, *journal*, and *text span* fields to support zone-weighted ranking. Wildcard queries are handled via lexicon expansion rather than a permuterm index to avoid excessive storage overhead.

Query Processing. The system supports Boolean, phrase, proximity, and wildcard queries. Queries are ranked using BM25 as the primary retrieval model, with TF-IDF available as an alternative. Query expansion is performed using the

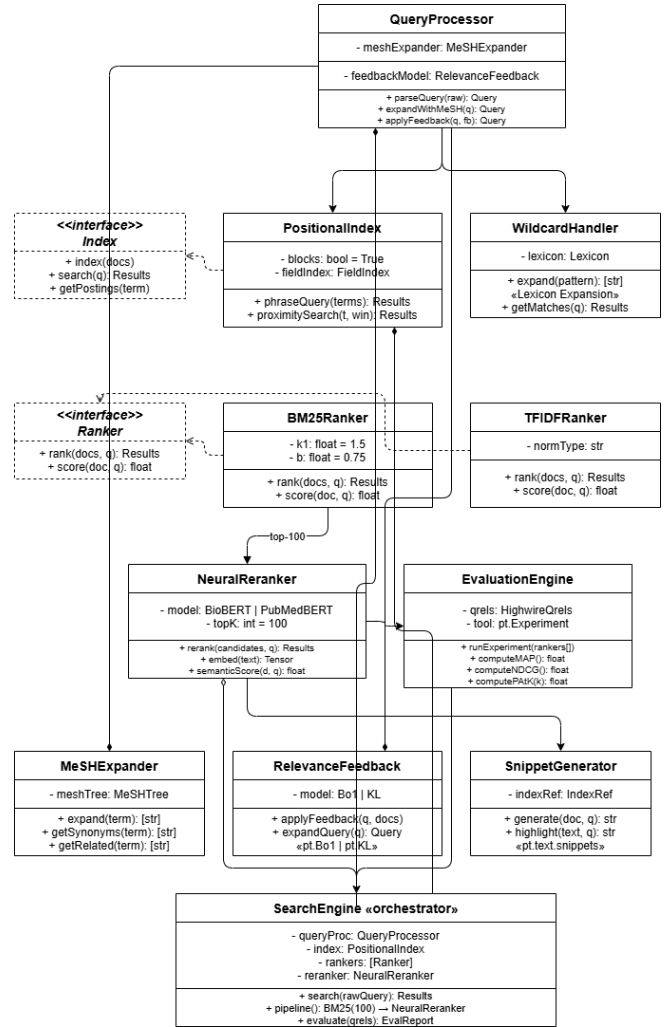


Figure 3. UML class diagram of the system components. *SearchEngine* acts as the orchestrator; *NeuralReranker* processes the top-100 BM25 candidates.

Medical Subject Headings (MeSH) vocabulary [5] to improve queries with synonyms and related biomedical terms, improving recall over domain-specific terminology. Relevance feedback is applied using PyTerrier’s built-in *pt.rewrite.Bo1()* and *pt.rewrite.KL()* models, which expand the query based on top-ranked feedback documents.

Ranking. The retrieval pipeline (see Figure 2) first retrieves the top-100 candidate documents using BM25 with specified parameters. TF-IDF is available as an alternative first-stage ranker for comparison. The *NeuralReranker* then scores each candidate using a pre-trained biomedical transformer (BioBERT or PubMedBERT) via cosine similarity between the query and document embeddings, producing the final ranked list.

User Interface. A lightweight interface (Search UI, Results page, Feedback UI, Eval UI) connects to the backend via

an API Gateway exposing `/search`, `/feedback`, and `/evaluate` endpoints. The interface supports phrase queries, wildcard queries, and proximity operators. Results are ranked and displayed with snippets generated by `pt.text.snippets(index_ref)` with query-term highlighting. Relevance feedback can be submitted via the Feedback UI to refine results. System performance is evaluated through the Eval UI using `pt.Experiment` with MAP, NDCG@K, P@K, MRR, and R-Prec metrics.

2.1 Milestones and Schedule

The schedule from the proposal is largely on track. The main structural change is the replacement of the Java implementation with PyTerrier, which has accelerated index construction and retrieval pipeline development. Neural re-ranking (BioBERT or PubMedBERT) has been added as a new milestone. The revised schedule is:

- Architecture design, data review, preprocessing (*done*)
- Index construction (positional + field) (*done*)
- Query processing: Boolean, phrase, proximity, wildcard (*done*)
- Ranking models (TF-IDF ranking & BM25), Progress Report, first evaluation (*done*)
- Query expansion (MeSH) & relevance feedback (Bo1/KL) (*1 week*)
- Neural re-ranking (BioBERT/PubMedBERT) (*1 week*)
- Snippet generation & UI integration (*1 week*)
- Final evaluation, benchmarking & report (*2 week*)

3 Experiments and Evaluation

3.1 Dataset

The Highwire Press medical document collection [1] has been acquired via the `ir_datasets` library. The corpus consists of 162,259 full-text scientific articles from 49 biomedical journals, covering the TREC Genomics Tracks of 2006 and 2007. The total number of words across all documents is approximately 994 million, with an average of 6,126 words per document. After PyTerrier tokenization, which includes lowercasing, punctuation splitting, and separation of alphanumeric tokens, the corpus yields approximately 1.06 billion tokens, with an average of 6,542 tokens per document. Each document contains structured fields including `doc_id`, `journal`, `title`, and `text spans`. For evaluation, we use the 64 queries (28 from 2006, 36 from 2007) and their corresponding official TREC relevance judgments (qrels) as ground truth.

3.2 Inverted index

The IR system is built using PyTerrier [4] based on the feedback, with the `ir_datasets` library used for corpus and qrels access. The inverted index was constructed over the full Highwire corpus using `pt.IterDictIndexer` with `blocks=True` to enable positional indexing for phrase and proximity queries. The index covers all 162,259 documents and contains approximately 1.06 billion tokens.

A first version of the IR system has been implemented and evaluated. Two ranking models are currently available: TF-IDF and BM25, both implemented using `pt.terrier.Retriever`. Experiments are conducted using `pt.Experiment` with the official TREC qrels as ground truth, covering all 64 queries from the TREC Genomics 2006–2007 tracks. Evaluation metrics used are MAP, R-Precision, MRR, P@5, P@10, and NDCG@10, all of which were covered in class.

To investigate BM25 parameter sensitivity on this corpus, we tested multiple configurations of k_1 and b . Since the Highwire corpus consists of full-text scientific articles with an average of 6,542 tokens per document, we hypothesized that BM25’s length normalization parameter b may be penalizing relevant documents that are long by nature rather than by topic drift. We therefore tested lower values of b and higher values of k_1 to reduce this effect. Table 1 shows the results.

Table 1. Ranking performance on Highwire qrels (64 queries, TREC Genomics 2006–2007).

Model	MAP	R-Prec	MRR	P@5	P@10	NDCG@10
TF-IDF	0.1516	0.1792	0.4140	0.2594	0.2172	0.2459
BM25 ($k_1=1.5$, $b=0.75$)	0.1039	0.1138	0.3003	0.1500	0.1422	0.1584
BM25 ($k_1=1.5$, $b=0.50$)	0.1036	0.1177	0.2966	0.1625	0.1469	0.1604
BM25 ($k_1=1.5$, $b=0.25$)	0.1048	0.1167	0.3103	0.1625	0.1438	0.1593
BM25 ($k_1=1.5$, $b=0.00$)	0.1036	0.1196	0.2943	0.1438	0.1375	0.1508
BM25 ($k_1=2.0$, $b=0.25$)	0.1027	0.1215	0.3170	0.1594	0.1500	0.1656
BM25 ($k_1=3.0$, $b=0.25$)	0.1002	0.1166	0.3312	0.1781	0.1516	0.1701

TF-IDF outperforms all BM25 configurations across most metrics. Contrary to our expectation, reducing b toward 0 did not close the gap with TF-IDF. Increasing k_1 improved MRR and P@5 marginally but affected MAP. This suggests that BM25’s length normalization is disadvantageous for this corpus, and that no parameter tuning is sufficient to match TF-IDF performance here. The remaining ranking schemes, including BM25 with MeSH-based query expansion and neural re-ranking will be evaluated and reported in the final submission.

3.3 Things to do

- Complete MeSH-based query expansion and measure its effect on recall over the BM25 baseline.
- Integrate relevance feedback (Bo1 / KL) into the retrieval pipeline and evaluate its impact on evaluation metrics.
- Implement the neural re-ranking stage using BioBERT or PubMedBERT and evaluate against BM25 results.
- Integrate `pt.text.snippets()` into the UI results page.
- Run the full `pt.Experiment` comparison across all ranking schemes (TF-IDF, BM25, BM25+MeSH, BM25+Neural) and measure their performance using evaluation metrics that we discussed in class.
- Prepare the final report and presentation.

References

- [1] [n.d.]. Highwire Press Medical Document Collection, TREC Genomics Track 2006–2007. <https://ir-datasets.com/highwire.html>.
- [2] Yu Gu, Robert Tinn, Hao Cheng, Michael Lucas, Naoto Usuyama, Xiaodong Liu, Jianfeng Tang, Hoifung Poon, and Jianfeng Gao. 2021. Domain-Specific Language Model Pretraining for Biomedical Natural Language Processing. <https://arxiv.org/abs/2007.15779>.
- [3] Jinhyuk Lee, Wonjin Yoon, Sungdong Kim, Donghyeon Kim, Sunkyu Kim, Chan Ho So, and Jaewoo Kang. 2019. BioBERT: a pre-trained biomedical language representation model for biomedical text mining. <https://arxiv.org/abs/1901.08746>.
- [4] Craig Macdonald and Nicola Tonellotto. 2020. Declarative Experimentation in Information Retrieval using PyTerrier. In *Proc. of the 2020 ACM SIGIR International Conference on Theory of Information Retrieval (ICTIR)*. ACM.
- [5] National Library of Medicine. [n.d.]. Medical Subject Headings (MeSH). <https://www.ncbi.nlm.nih.gov/mesh/>.